# Exercice 1

Nous désirons développer un programme pour la gestion (très simplifiée) d'un parc de véhicules destinés à la location.

Chaque véhicule doit renseigner le *modèle*, *l'année d'achat* (un entier), le *prix d'achat* (capable de gérer les centimes), le *numéro d'immatriculation* et le *permis* (une lettre) nécessaire à la conduite de ce véhicule.

Plusieurs types de véhicules sont possibles à la location, certains ayant des caractéristiques particulières. Ainsi, il est demandé de représenter l'information comme quoi une voiture dispose ou non d'un *autoradio*. Pour la location d'un camion, le *volume* de stockage possible du camion doit être précisé. Enfin, des autocars peuvent aussi être loués, et pour ces véhicules il est nécessaire de connaître leur *volume* (pas forcément entier) de stockage possibles ainsi que le *nombre de passagers* qu'ils peuvent transporter.

## *Codage des méthodes*

1. Définir un constructeur qui initialise les différents champs de chacune des classes.
2. Écrire les méthodes `ajouterAutoradio()` et `enleverAutoradio()` qui permettent de fixer la valeur de l'attribut autoradio d'une voiture.
3. Écrire la méthode `age()` qui retourne l'âge d'un véhicule (en nombre d'années). L'année actuelle sera écrite en dur dans le code.
4. Écrire la méthode `afficherVehicule()` qui affiche sur la sortie standard (en utilisant la méthode **`System.`***`out`***`.println(…)`**) une chaîne de caractères caractérisant un véhicule. Écrire la méthode `afficherVoiture()` qui affiche une chaîne de caractères caractérisant une voiture.
5. Écrire une méthode `peutTransporterVolume(…)` qui, à partir d'un volume donné en paramètre, retourne vrai ou faux selon que le camion peut ou non permettre le transport de ce volume.
6. Écrire une méthode `coutLocation(…)` qui calcule le coût quotidien de location d'un véhicule. Ce coût est calculé comme suit : pour les véhicules de moins d'un an, le coût de location est le prix d'achat du véhicule / 200, et pour les véhicules plus anciens, le coût de location est le prix d'achat / 250.
7. Écrire une méthode `peutTransporterPassagers(…)` dans la classe adéquate qui, à partir d'un nombre de passagers et d'un volume moyen par passager (représentant les bagages), retourne vrai ou faux selon que l'autocar peut ou non transporter ces passagers.
8. Écrire la méthode `afficherCamion()` qui affiche une chaîne de caractères caractérisant un camion.
9. Écrire la méthode `afficherAutocar()` qui affiche une chaîne de caractères caractérisant un autocar.

## *Classe contenant le programme principal*

1. Écrire/compléter un programme de test créant une (instance de la classe) voiture Twingo, achetée cette année 10000 €, immatriculée AB-123-CD, avec un autoradio et nécessitant un permis B. Puis afficher la chaîne de caractères représentant la voiture. Le résultat est-il celui escompté ?
2. Créer un camion J9, acheté 20000€ en 2008, immatriculé 1234-AB-56, utilisable avec un permis B, et pouvant transporter 25 m³. Afficher sur la sortie standard la chaîne de caractères représentant le camion. Afficher si ce camion peut transporter 7 m³, on utilisera pour cela un appel à la méthode `peutTransporterVolume(…)`.
3. Afficher le coût de location de la Twingo précédemment créée, ainsi que celui du camion de type J9.
4. Créer un autocar de type FRI, acheté 90000€ en 2005, immatriculé 123-AB-45, nécessitant un permis D, permettant de transporter 53 passagers et disposant d'une soute à bagages de 3 m³. Affichez si cet autocar peut transporter 40 passagers ayant chacun 0,1 m³ de bagages.
5. Les classes de l'application sont appelées à être complétées et utilisées par plusieurs programmeurs. Quelle solution proposez-vous, en ce qui concerne les champs (attributs et méthodes), pour assurer cette gestion ? Implémentez-là.